

Lesson 23 — Decomposing a Big Program into Functions

Matches LEGO Teacher Portal lesson: “Art Designer”.

Objective (I can...): I can break a larger program into several well-named functions that work together.

CS concept: Decomposition via functions; modular design · **Builds on:** Lesson 22 (parameters) and Lesson 3 (decomposition).

AI Connection (2 min): A real AI pipeline is many functions chained — capture, preprocess, classify, respond — each testable on its own. *First-principles question:* “Why is a program made of small named functions easier to fix than one giant block?”

Materials: LEGO CS & AI kit + device per pair, plan sheets, notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): “Carve it up” — pairs take a described multi-stage behavior and split it into 3–4 named functions on paper, deciding what each owns. - **Concept + demo (10):** Demo a main program that calls three functions in order (e.g., “startup()”, “patrol(distance)”, “shutdown()”); show the main reads like an outline. Emphasize each function is tested alone before assembly. - **Build/Code (30):** Pairs plan a behavior as 3 functions, build and test each function independently, then assemble a short main that calls them. They log each function's single job in the notebook. *Swap at 15 min.* - **Challenge (7):** Reuse one function in two different places in the main to prove its generality. - **Share + pack-up (5):** Pairs read their main program as an outline of function calls and explain how modularity made testing easier. Model stays built.

Pair roles: Each partner owns and tests one or two functions, then they assemble the main together; swap at 15 min. (*Trio: three functions, one each, assemble together.*)

Differentiation: *Support* — provide two function stubs to fill; students write the third and the main. *Extension / fast-finisher* — four functions, one parameterized. *Reteach* — build and test one function fully, then add the next, assembling incrementally.

Exit check: Pair's program is organized into 3+ working, well-named functions called from a readable main. **At-home (optional):** Outline a project (cleaning your room) as a main that calls 4 named functions; define one. ~10 min.